Project Title

A Project Report

submitted in partial fulfillment of the requirements

of

AICTE Internship on AI: Transformative Learning
with
TechSaksham – A joint CSR initiative of Microsoft & SAP

by

Khushi Dasar,

khushi.dasar.aiml22@ggits.net

Under the Guidance of

Pavan Sumohana

Master Trainer ,Edunet Foundation

ACKNOWLEDGEMENT

We would like to take this opportunity to express our deep sense of gratitude to all individuals who helped us directly or indirectly during this thesis work.

Firstly, we would like to thank my supervisor, Pavan Sumohana for being a great mentor and the best adviser I could ever have. His advice, encouragement and the critics are a source of innovative ideas, inspiration and causes behind the successful completion of this project. The confidence shown in me by him was the biggest source of inspiration for me. It has been a privilege working with him for the last one year. He always helped me during my project and many other aspects related to the program. His talks and lessons not only help in project work and other activities of the program but also make me a good and responsible professional.

ABSTRACT

1.1 Problem Statement:

In the modern recruitment process, organizations receive an overwhelming number of resumes for each job opening. Screening these resumes to identify the most suitable candidates is a time-consuming and labor-intensive task. Automating this process using machine learning and natural language processing (NLP) techniques can significantly improve the efficiency and effectiveness of recruitment.

1.2 Objective:

The objective of this project is to develop an automated resume screening system using Machine Learning (ML) and Natural Language Processing (NLP) techniques. The system aims to analyze and filter resumes based on predefined criteria such as skills, experience, education, and job relevance. By automating the resume screening process, the solution will:

- Reduce manual effort and time consumption in candidate shortlisting.
- Improve accuracy by identifying the most relevant candidates based on job descriptions.
- Eliminate biases by providing a standardized and objective screening process.
- Enhance recruitment efficiency, enabling HR teams to focus on high-quality candidates.

1.3 Methodology:

The development of an **Automated Resume Screening System** using **Machine Learning (ML)** and **Natural Language Processing (NLP)** follows a structured methodology, integrating Streamlit for the frontend, Python for the backend, and key frameworks such as **Scikit-learn, NLTK, and SpaCy** for text processing and classification.

---

1. <u>Software Requirements-</u>

- **Frontend:** Streamlit (for an interactive web-based user interface).
- **Backend:** Python (for implementing ML and NLP models).
- **Frameworks:**
  - **Scikit-learn (Sklearn):** For feature extraction and similarity measurement.
  - **NLTK & SpaCy:** For NLP tasks like text parsing, tokenization, and named entity recognition.
- **Deployment:** Streamlit Cloud (for hosting and accessing the system online).

---

2. <u>Data Collection & Preprocessing-</u>

- Collect resumes in **PDF, DOCX, TXT** formats.
- Extract job descriptions for different roles.
- Use **NLTK & SpaCy** for:

- **Text Extraction:** Convert resumes into raw text.
- **Text Cleaning:** Remove stopwords, special characters, and unwanted spaces.
- **Tokenization & Lemmatization:** Break text into words and convert them to their base forms.
- **Named Entity Recognition (NER):** Extract relevant information like skills, education, and experience.

---

3. <u>Feature Engineering & Representation-</u>

- Convert text into numerical form for ML models using **Scikit-learn**:
  - **TF-IDF (Term Frequency-Inverse Document Frequency):** Extract important keywords.
  - **Word Embeddings (Word2Vec, SpaCy, or BERT):** Capture the semantic meaning of resumes.
- Calculate **text similarity scores** between resumes and job descriptions using:
  - **Cosine Similarity**
  - **Jaccard Similarity**
- Extract structured information like **skills, experience, education, and certifications**.

---

4. <u>Machine Learning Model for Resume Classification-</u>

- Train a classification model to categorize resumes based on relevance using **Scikit-learn**:
  - **Logistic Regression**
  - **Random Forest**
  - **Support Vector Machines (SVM)**
- Score and rank resumes based on:
  - **Skill Match Percentage**
  - **Experience Relevance**
  - **Education Suitability**

---

5. <u>Building the Frontend with Streamlit-</u>

- Develop an **interactive UI** using **Streamlit**:
  - Upload resumes and job descriptions.
  - Display ranked candidates with a **resume match score**.
  - Visualize skill relevance using bar charts and similarity heatmaps.

---

6. <u>Deployment Using Streamlit Cloud-</u>

- Deploy the system using **Streamlit Cloud** for easy access.
- Ensure real-time resume screening and ranking for recruiters.
- Allow users to **upload resumes**, **view shortlisted candidates**, and **download reports**.

---

7. <u>Performance Evaluation & Optimization-</u>

- Measure model accuracy using **Precision, Recall, and F1-score**.
- Optimize results using **hyperparameter tuning**.
- Collect HR feedback to improve the resume screening algorithm iteratively.

1.4 Key Results:

- **Reduction** in manual resume screening time.
- **Accuracy** in candidate matching using ML & NLP techniques.
- **Instant ranking & visualization** via a **Streamlit-based UI**.
- **Deployed on Streamlit Cloud** for easy access & bulk processing.
- **Bias-free screening** by focusing on **skills & qualifications**.
- **High performance**

1.5 Summary of the project:

Conclusion-

- **Functional Solution**: The project developed a Flask-based web application that effectively ranks and classifies resumes using machine learning models and text processing techniques streamlining the hiring process by matching resumes to job descriptions.
- **User-Friendly Interface:** It features an intuitive interface for uploading resumes, inputting job descriptions, and viewing ranked results, with responsive design and clear error handling
- **Data Processing and Ranking**: The system utilizes TF-IDF vectorization and cosine similarity to evaluate resume relevance providing a quantitative measure of how well each resume aligns with the job description .
- **Future Enhancements**: Potential improvements include integrating advanced NLP models, adding more features for nuanced ranking, and expanding the data processing pipeline to enhance the system's accuracy and adaptability.

TABLE OF CONTENT

Abstract ..... **I**

Chapter 1. Introduction **1**

1.1 Problem Statement 1

1.2 Motivation 1

1.3 Objectives 2

1.4. Scope of the Project 2

Chapter 2. Literature Survey **3**

Chapter 3. Proposed Methodology **5**

Chapter 4. Implementation and Results **7**

Chapter 5. Discussion and Conclusion **8**

References 9

CHAPTER 1

Introduction

1.6 Problem Statement:

In the modern recruitment process, organizations receive an overwhelming number of resumes for each job opening. Screening these resumes to identify the most suitable candidates is a time-consuming and labor-intensive task. Automating this process using machine learning and natural language processing (NLP) techniques can significantly improve the efficiency and effectiveness of recruitment.

1.7 Motivation:

Why Was This Project Chosen?

This project was chosen due to the increasing challenges in modern recruitment, where **HR teams struggle with manually screening large volumes of resumes**. Traditional hiring processes are:

- **Time-consuming:** Sorting through hundreds of applications delays hiring decisions.
- **Prone to bias:** Subjective evaluation can lead to unfair candidate selection.
- **Error-prone:** Human errors may result in overlooking qualified candidates.
    - By leveraging **Machine Learning (ML) and Natural Language Processing (NLP)**, this project **automates and optimizes resume screening**, ensuring faster and more accurate hiring.

Potential Applications-

- **Corporate Hiring:** Automates resume filtering for HR departments in large organizations.
- **Job Portals & Recruitment Agencies:** Enhances candidate-job matching accuracy.
- **University Placements:** Helps educational institutions shortlist students for internships and jobs.
- **Freelance & Gig Platforms:** Matches freelancers with the most relevant projects based on skills.

Impact of This Project-

- Helps companies hire the best talent faster.
- Streamlines recruitment, saving time and effort for HR teams.
- Ensures a fair, transparent, and efficient hiring process.

1.8 Objective:

The objective of this project is to **automate the resume screening process** using **Machine Learning (ML) and Natural Language Processing (NLP)** techniques. The system aims to:

- **Reduce manual effort** by automatically analyzing and filtering resumes.
- **Improve hiring accuracy** by matching candidates' skills, experience, and qualifications with job descriptions.
- **Ensure unbiased recruitment** through data-driven evaluation, minimizing human bias.
- **Enhance efficiency** by significantly reducing the time required for candidate shortlisting.
- **Provide a user-friendly interface** using **Streamlit**, allowing HR teams to upload resumes and receive ranked recommendations instantly.

1.9 Scope of the Project:

This project focuses on automating **resume screening** using **Machine Learning (ML) and Natural Language Processing (NLP)** to enhance recruitment efficiency.

- **Automated Resume Processing:** Extracts key details (skills, experience, education) from resumes in **PDF, DOCX, and TXT** formats.
- **Job-Resume Matching:** Uses NLP to compare resumes with job descriptions and rank candidates based on relevance.
- **Machine Learning Models:** Implements **Scikit-learn, NLTK, and SpaCy** for **text analysis, feature extraction, and classification**.
- **User-Friendly Interface:** Developed with **Streamlit**, allowing recruiters to **upload resumes and get instant candidate rankings**.
- **Cloud-Based Deployment:** Hosted on **Streamlit Cloud** for accessibility and scalability.

Limitations of the Project-

- Limited Resume Formats
- Language Constraints
- Contextual Understanding
- Dependency on Training Data
- No Final Decision-Making

CHAPTER 2

Literature Survey

2.1 Review relevant literature or previous work in this domain.

Existing research highlights the inefficiencies of manual resume screening, emphasizing the need for AI-driven automation. Studies show that ML and NLP techniques improve accuracy, efficiency, and fairness in recruitment.

Key Findings:

- **Text Mining & ML Models**: Techniques like TF-IDF, Word2Vec, and BERT improve resume-job matching.
- **NER-Based Resume Parsing:** SpaCy & NLTK effectively extract skills, education, and experience.
- **Real-World Applications:** Companies like LinkedIn, Amazon, and HireVue use AI for recruitment, reducing time-to-hire by 40%.

2.2 Mention any existing models, techniques, or methodologies related to the problem.

Existing Models, Techniques, and Methodologies

Several ML and NLP techniques are used for automated resume screening-

Machine Learning Models:

- Logistic Regression, SVM, Random Forest – Used for classifying resumes as suitable or not.
- BERT, LSTM – Deep learning models for semantic understanding and resume-job matching.

NLP Techniques:

- TF-IDF, Word2Vec – Extracts key features from resumes.
- NER (SpaCy, NLTK) – Identifies skills, education, and experience.
- Cosine Similarity – Measures relevance between resumes and job descriptions.

Existing Systems:

- LinkedIn & HireVue – AI-driven job matching.
- ATS (Workday, Taleo) – Uses keyword-based filtering for resume screening.

2.3 Highlight the gaps or limitations in existing solutions and how your project will address them.

Gaps in Existing Systems:

- Keyword-Based Filtering: Many ATS rely on exact keyword matches, missing qualified candidates.
- Bias Issues: Some AI models favor certain demographics due to biased training data.

- Poor Contextual Understanding: Traditional models fail to grasp resume-job relevance beyond simple text matching.
- Limited Resume Formats: Many systems struggle with PDFs, scanned documents, and unstructured formats.
- Complex & Costly: Existing solutions often require HR software integration, making them less accessible.

How This Project Solves These Issues:

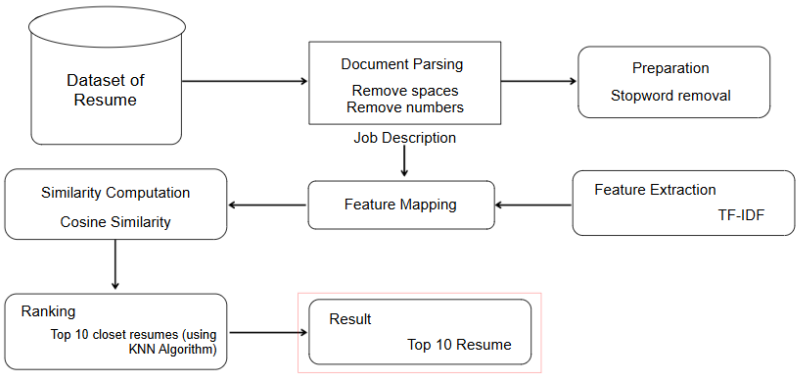
- Semantic Matching: Uses TF-IDF, Word2Vec, and BERT to match resumes based on meaning, not just keywords.
- Bias Reduction: Focuses on skills and experience, ignoring personal attributes to ensure fair screening.
- Advanced NLP Parsing: Utilizes SpaCy & NLTK to extract skills, education, and experience accurately.
- Supports Multiple Formats: Works with PDF, DOCX, and TXT, improving compatibility.
- User-Friendly & Scalable: Built on Streamlit, enabling real-time resume ranking without complex setups.

CHAPTER 3

Proposed Methodology

3.1 System Design

Diagram of the Proposed Solution



The provided diagram represents the workflow of the automated resume screening system. Below is a detailed explanation of each step:

Step 1: Dataset of Resumes

- The system begins with a dataset of resumes in various formats (PDF, DOCX, TXT).
- These resumes are stored in a structured way for further processing.

Step 2: Document Parsing

- Raw resumes are cleaned and preprocessed:
- Removing spaces
- Removing numbers
- Text normalization

Step 3: Preparation

Additional text preprocessing is performed:

- Removing stopwords (e.g., "the," "is," "and") to focus on meaningful words.
- Lowercasing text for consistency.

Step 4: Feature Extraction (TF-IDF)

- TF-IDF (Term Frequency-Inverse Document Frequency) is used to convert resume text into numerical vectors.
- It helps in identifying important words based on their relevance in a document rather than just frequency.

Step 5: Feature Mapping

- The processed resumes and job descriptions are mapped into a numerical space.
- This allows for direct comparison between resumes and job descriptions.

Step 6: Similarity Computation (Cosine Similarity)

- Cosine Similarity is used to measure how similar each resume is to the job description.
- A higher similarity score indicates a better match.

Step 7: Ranking (KNN Algorithm)

- K-Nearest Neighbors (KNN) Algorithm ranks resumes based on their similarity scores.
- The top 10 closest resumes to the job description are selected.

Step 8: Result (Top 10 Resumes)

The system outputs the Top 10 best-matching resumes, which can be reviewed by recruiters.

3.2 Requirement Specification

Mention the tools and technologies required to implement the solution.

3.2.1 Hardware Requirements:

To run the AI-based Resume Screening System, the following hardware specifications are recommended:

For Development & Training:

- Processor: Intel i5/i7 (10th Gen or higher) / AMD Ryzen 5/7
- RAM: Minimum 8GB (16GB recommended for faster processing)
- Storage: At least 256GB SSD (512GB+ recommended for handling datasets)
- GPU: NVIDIA GPU (optional, but recommended for deep learning models)

For Deployment & Hosting:

- Cloud Server (e.g., AWS, Google Cloud, Azure) or Local Machine
- CPU: 4+ cores (Intel Xeon / AMD EPYC for better performance)
- RAM: Minimum 8GB (higher for concurrent requests)
- Storage: 50GB+ SSD for model storage and resume dataset
- GPU (Optional): Required if using BERT or deep learning models

3.2.2 Software Requirements:

- Frontend: Streamlit
- Backend: Python
- Frameworks: Sklearn/ NLTK / Spacy

- Scikit-learn (Sklearn): For extracting text features and finding the similarity between the documents.
- NLTK and SpaCy: For natural language processing tasks such as text parsing, tokenization.
- Deployment: Deployment using Streamlit cloud

CHAPTER 4

Implementation and Result

4.1 Snap Shots of Result:

```
[ ] !pip install streamlit
    !pip install PyPDF2
```

```
import streamlit as st
from PyPDF2 import PdfReader
import pandas as pd
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity
```

```
[ ] # Function to extract text from PDF
    def extract_text_from_pdf(file):
        pdf = PdfReader(file)
        text = ""
        for page in pdf.pages:
            text += page.extract_text()
        return text
```

```
# Combine job description with resumes
# Function to rank resumes based on job description
def rank_resumes(job_description, resumes):
    # Combine job description with resumes
    documents = [job_description] + resumes
    vectorizer = TfidfVectorizer().fit_transform(documents)
    vectors = vectorizer.toarray()

    # Calculate cosine similarity
    job_description_vector = vectors[0]
    resume_vectors = vectors[1:]
    cosine_similarities = cosine_similarity([job_description_vector], resume_vectors).flatten()
```

```
[ ] # Combine job description with resumes
    # Function to rank resumes based on job description
    def rank_resumes(job_description, resumes):
        # Combine job description with resumes
        documents = [job_description] + resumes
        vectorizer = TfidfVectorizer().fit_transform(documents)
        vectors = vectorizer.toarray()

        # Calculate cosine similarity
        job_description_vector = vectors[0]
        resume_vectors = vectors[1:]
        cosine_similarities = cosine_similarity([job_description_vector], resume_vectors).flatten()

        return cosine_similarities
        job_description_vector = vectors[0]
        resume_vectors = vectors[1:]
        cosine_similarities = cosine_similarity([job_description_vector], resume_vectors).flatten()

        return cosine_similarities
```

```
[ ] # Streamlit app
    st.title("AI Resume Screening & Candidate Ranking System")
    # Job description input
    st.header("Job Description")
    job_description = st.text_area ("Enter the job description")

2025-03-24 03:40:09.098 WARNING streamlit.runtime.scriptrunner_utils.script_run_context: Thread 'MainThread': missing ScriptRunContext! This warning can be ignored when running in bare
2025-03-24 03:40:09.105
  Warning: to view this Streamlit app on a browser, run it with the following
  command:
```

```
[ ] # Job description input
    st.header("Job Description")
    job_description = st.text_area ("Enter the job description")

2025-03-24 03:40:09.098 WARNING streamlit.runtime.scriptrunner_utils.script_run_context: Thread 'MainThread': missing ScriptRunContext! This warning can be ignored when running in bare
2025-03-24 03:40:09.105
  Warning: to view this Streamlit app on a browser, run it with the following
  command:

    streamlit run /usr/local/lib/python3.11/dist-packages/colab_kernel_launcher.py [ARGUMENTS]
2025-03-24 03:40:09.106 Thread 'MainThread': missing ScriptRunContext! This warning can be ignored when running in bare mode.
2025-03-24 03:40:09.108 Thread 'MainThread': missing ScriptRunContext! This warning can be ignored when running in bare mode.
2025-03-24 03:40:09.109 Thread 'MainThread': missing ScriptRunContext! This warning can be ignored when running in bare mode.
2025-03-24 03:40:09.110 Thread 'MainThread': missing ScriptRunContext! This warning can be ignored when running in bare mode.
2025-03-24 03:40:09.111 Thread 'MainThread': missing ScriptRunContext! This warning can be ignored when running in bare mode.
2025-03-24 03:40:09.112 Thread 'MainThread': missing ScriptRunContext! This warning can be ignored when running in bare mode.
2025-03-24 03:40:09.113 Thread 'MainThread': missing ScriptRunContext! This warning can be ignored when running in bare mode.
2025-03-24 03:40:09.114 Session state does not function when running a script without `streamlit run`
2025-03-24 03:40:09.115 Thread 'MainThread': missing ScriptRunContext! This warning can be ignored when running in bare mode.
2025-03-24 03:40:09.116 Thread 'MainThread': missing ScriptRunContext! This warning can be ignored when running in bare mode.
```

```
# File uploader
st.header("Upload Resumes")
uploaded_files = st.file_uploader("Upload PDF files", type=["pdf"], accept_multiple_files=True)

2025-03-24 03:40:16.293 Thread 'MainThread': missing ScriptRunContext! This warning can be ignored when running in bare mode.
2025-03-24 03:40:16.295 Thread 'MainThread': missing ScriptRunContext! This warning can be ignored when running in bare mode.
2025-03-24 03:40:16.296 Thread 'MainThread': missing ScriptRunContext! This warning can be ignored when running in bare mode.
2025-03-24 03:40:16.297 Thread 'MainThread': missing ScriptRunContext! This warning can be ignored when running in bare mode.
2025-03-24 03:40:16.298 Thread 'MainThread': missing ScriptRunContext! This warning can be ignored when running in bare mode.
2025-03-24 03:40:16.298 Thread 'MainThread': missing ScriptRunContext! This warning can be ignored when running in bare mode.
2025-03-24 03:40:16.300 Thread 'MainThread': missing ScriptRunContext! This warning can be ignored when running in bare mode.
```

```
[ ] if uploaded_files and job_description:
        st.header("Ranking Resumes")

        resumes = []
        for file in uploaded_files:
            text = extract_text_from_pdf(file)
            resumes.append(text)

        # Rank resumes
        scores = rank_resumes(job_description, resumes)

        # Display scores
        results = pd.DataFrame({"Resume": [file.name for file in uploaded_files], "Score": score })
        results = results.sort_values(by="Score", ascending=False)

        st.write(results)
```

CHAPTER 5

Discussion and Conclusion

5.1 Future Work:

Future Scope

- Integrate advanced NLP techniques like BERT or GPT for improved contextual understanding and semantic matching
- Implement a custom-trained deep learning model for more accurate resume ranking.
- Incorporate structured data extraction from resumes to better assess qualifications and skills
- Expand with interactive dashboards for data visualization and improved user interaction
- Add multi-language support to cater to a global audience.
- Strengthen security measures for robust protection of data and system integrity
- Establish regular updates and feedback loops for continuous improvement and adaptability to industry changes

5.2 Conclusion:

- **Functional Solution:** The project developed a Flask-based web application that effectively ranks and classifies resumes using machine learning models and text processing techniques streamlining the hiring process by matching resumes to job descriptions.
- **User-Friendly Interface:** It features an intuitive interface for uploading resumes, inputting job descriptions, and viewing ranked results, with responsive design and clear error handling

- **Data Processing and Ranking**: The system utilizes TF-IDF vectorization and cosine similarity to evaluate resume relevance, providing a quantitative measure of how well each resume aligns with the job description
- **Future Enhancements:** Potential improvements include integrating advanced NLP models, adding more features for nuanced ranking, and expanding the data processing pipeline to enhance the system's accuracy and adaptability

REFERENCES

[1]. Ming-Hsuan Yang, David J. Kriegman, Narendra Ahuja, "Detecting Faces in Images: A Survey", IEEE Transactions on Pattern Analysis and Machine Intelligence, Volume. 24, No. 1, 2002.